

Cover

WEGZ Credit Union Security Shield Report

Elizabeth Culbertson, Zina Dupal, Graham Kempiners, and Westin Walrath



Overview

Overview

In the past, bank security was mostly heavy vault doors and security cameras. Today, institutions like WEGZ Credit Union face a more invisible threat: “Social Engineering.” Scammers now use AI to clone the exact voice of a manager or create fake videos of executives. They use these “deepfakes” to trick employees into moving money or sharing passwords. Because these scams target human psychology rather than just computer software, we needed a database that protects our people just as much as our servers.

Our solution, the Security Shield, acts as the credit union’s central “nervous system.” For example, if an employee receives a suspicious call that sounds like a member but feels “off,” they log it into the database immediately. Because the database links employees to these incident reports, it instantly knows exactly which branch is being targeted. If the threat is marked as “Severity Level 3,” the database triggers a “digital alarm,” sending a warning to the IT department. This ensures that a scam attempted at one branch becomes an instant warning for the entire organization, preventing any single location from being a “weak link.”

To keep the database running smoothly, we created two tools for managers: a web app and SQL agent. The web app allows managers to easily add new hires or remove employees who have left the union. This keeps our staff list accurate, which is paramount for knowing who is responsible for which computer and who has finished their security training. The SQL agent is like a digital assistant that understands English. Instead of writing complex Python code, a manager can simply ask, “Who hasn’t finished the voice-cloning training yet?” or “Show me the threats from this morning.” The agent finds the data and provides instant answers, making high-level security oversight easy for everyone.

At WEGZ Credit Union, we believe that an informed employee is the best defense. The Security Shield manages tracked security classes. When an employee completes a module—such as “How to Spot a Cloned Voice”—the database automatically updates their file. If the database sees a spike in a specific type of scam, managers can check the database to see which departments are behind on their training. This allows them to bridge those “vulnerability gaps” before a real attack happens.

A security database is only effective if you know who is using which tools. The database tracks every computer and device by its serial number, so we know exactly what equipment the credit union owns. The database records every login attempt. If someone tries to log in from a strange country or fails their password multiple times, the database “flags” that device for a manager to check on immediately, stopping unauthorized access before it starts.

Database Operations

When a new hire joins the credit union, the database executes a transaction that adds the individual to the EMPLOYEES roster, assigns them to a branch (via the EMPLOYEE_DEPARTMENT table), and links their specific hardware (like a laptop) from the ASSETS table to their EmployeeID. This ensures every person is immediately accountable for their assigned equipment.

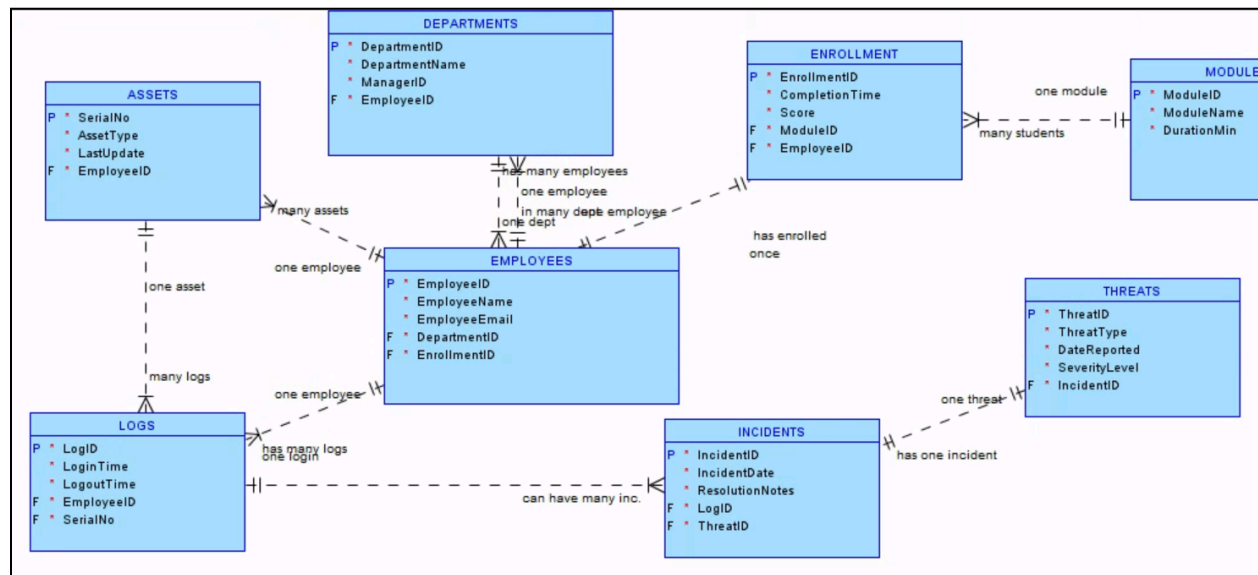
The database facilitates the security validation process by creating a transaction in the ENROLLMENT table. This links an employee to a specific MODULE (e.g., “Deepfake Voice Recognition”). Once the employee completes the module, the database updates the CompletionDate and Score, allowing the manager to check in real-time if a staff member is qualified to handle sensitive member information.

When a threat is detected, the database processes an operational transaction that captures the event details in the INCIDENTS table. This record is automatically linked to the system LOG entry where

the activity was flagged. If the THREATS severity is logged as a 4 or 5, the database provides the link for the IT department to review the ResolutionNotes and mitigate the risk across all branches.

Database Design and Implementation

Original Logical Model



These “blue entities” represent the major pieces of the WEGZ Credit Union—Employees, Departments, Assets, and Security Threats. As we developed this model, we were able to fully understand & flesh out the operation of our credit union, allowing us to build off of a foundation that truly made sense. A few of the struggles we ran into were related to the relationships that employees had with the rest of the company:

- Could an employee only enroll in one module?
- Could an employee be a member of multiple departments?
- Are log-in attempts manually entered by employees or automated through our program?

After deep discussion & further developing the inner workings of our organization, we were able to successfully answer all of the above questions. This process allowed us to mold & craft WEGZ Credit Union to be not only realistic, but to also align & make sense with the purpose of our database. Our discussion led us to the following conclusions on our original questions:

- An employee can enroll in many modules, which would maximize their comprehension of threatening behavior & incident risk mitigation.
- An employee can be a member of multiple departments; for example, a HR employee may be involved with a variety of departments in our organization.
- Log-in attempts (denoted in the LOGS table) are to be automated through our program, as both company assets and employees are connected to the table.

Business Rules

1. **EMPLOYEES to ASSETS (1:M)**: Each employee in the organization can be assigned one or many company assets (e.g., laptops). Each asset must be assigned to one employee to ensure a “chain” of custody. The cardinality is 1:1 on the Employee side, 1:M on the Asset side.
2. **EMPLOYEES to ENROLLMENT (1:M)**: An employee can be enrolled in one or many training modules (like Deepfake Awareness). Each enrollment record must belong to exactly one employee to track individual progress and scores. The cardinality is 1:1 on the Employee side, 1:M on the Enrollment side.

3. EMPLOYEES to LOGS (1:M): A single employee will have many access logs over time. Each log entry must be associated with one employee ID to maintain a secure audit log. The cardinality is 1:1 on the Employee side, 1:M on the Log side.
4. EMPLOYEES to EMPLOYEE_DEPARTMENT (1:M): An employee can be associated with one or more departments (e.g., an IT employee is also in the Marketing department). Each roster association must link back to one employee. The cardinality is 1:1 on the Employee side, 1:M on the Employee_Department side.
5. DEPARTMENTS to EMPLOYEE_DEPARTMENT (1:M): A department consists of one or many employee associations. Each association entry in the system must belong to one department. The cardinality is 1:1 on the Department side, 1:M on the Employee_Department side.
6. MODULES to ENROLLMENT (1:M): Each security training module (e.g., Module M001) can have many different employees enrolled in it. Each enrollment record must reference exactly one module. The cardinality is 1:1 on the Module side, 1:M on the Enrollment side.
7. LOGS to INCIDENTS (1:M): A system log entry can be flagged as a security incident (e.g., an unauthorized login attempt). While most logs have zero incidents, a single log entry could trigger multiple incident reports if multiple threats are detected. Each incident must be tied to one log. The cardinality is 1:1 on the Log side, 1:M on the Incident side.

Normalization Issues

Our objective was to eliminate data redundancy and ensure the database reached 3NF, or Third Normal Form. During the revisions of the initial logical model, we addressed the following issues:

1. The ModuleName and DepartmentName attributes presented potential 3NF violations; non-key attributes must depend only on the primary key, when fully normalized. We removed ModuleName to avoid redundancy, as ModuleID is the unique determinant (or primary key) for all things training modules. We decided to keep DepartmentName from the DEPARTMENTS table; while DepartmentID is the primary key, the name is necessary to distinguish between the same departments (e.g., “Marketing”) that may exist across different branches or locations.
2. We implemented the following changes to improve data integrity: We created an associative entity, EMPLOYEE_DEPARTMENT, to allow the database to track employees who may be a part of multiple departments at the same time, transitioning from 1:M to an M:M relationship; we changed the relationship between THREATS and INCIDENTS from 1:1 to 1:M, ensuring that a single threat type (e.g., “Deepfake Voice Scam”) can be related to multiple unique reports.
3. We reviewed DepartmentID and ModuleID and their necessity or if the names themselves could serve as identifiers. We determined that keeping the unique IDs is important because DepartmentNames (like “Accounting”) are not always unique and using numeric ID makes functional dependencies more stable; this protects the database from “Update Anomalies” if a department or module were ever renamed.

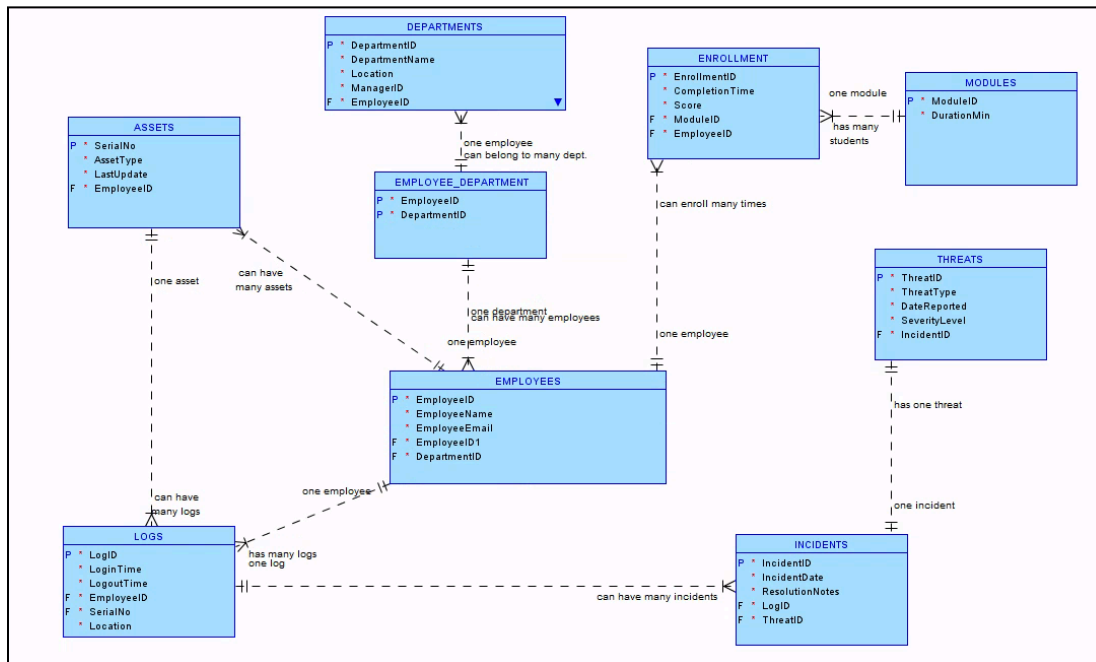
Initial Functional Dependency (FD) List:

- EMPLOYEES table: EmployeeID → EmployeeName, EmployeeEmail, DepartmentID
- DEPARTMENTS table: DepartmentID → DepartmentName, ManagerID
- ASSETS table: SerialNo → AssetType, LastUpdate, EmployeeID
- LOGS table: LogID → LoginTime, LogoutTime, EmployeeID
- THREATS table: ThreatID → ThreatType, SeverityLevel
- INCIDENTS table: IncidentID → IncidentDate, ResolutionNotes, LogID, ThreatID
- MODULES table: ModuleID → DurationMin
- ENROLLMENT table: (EmployeeID, ModuleID) → CompletionDate, Score

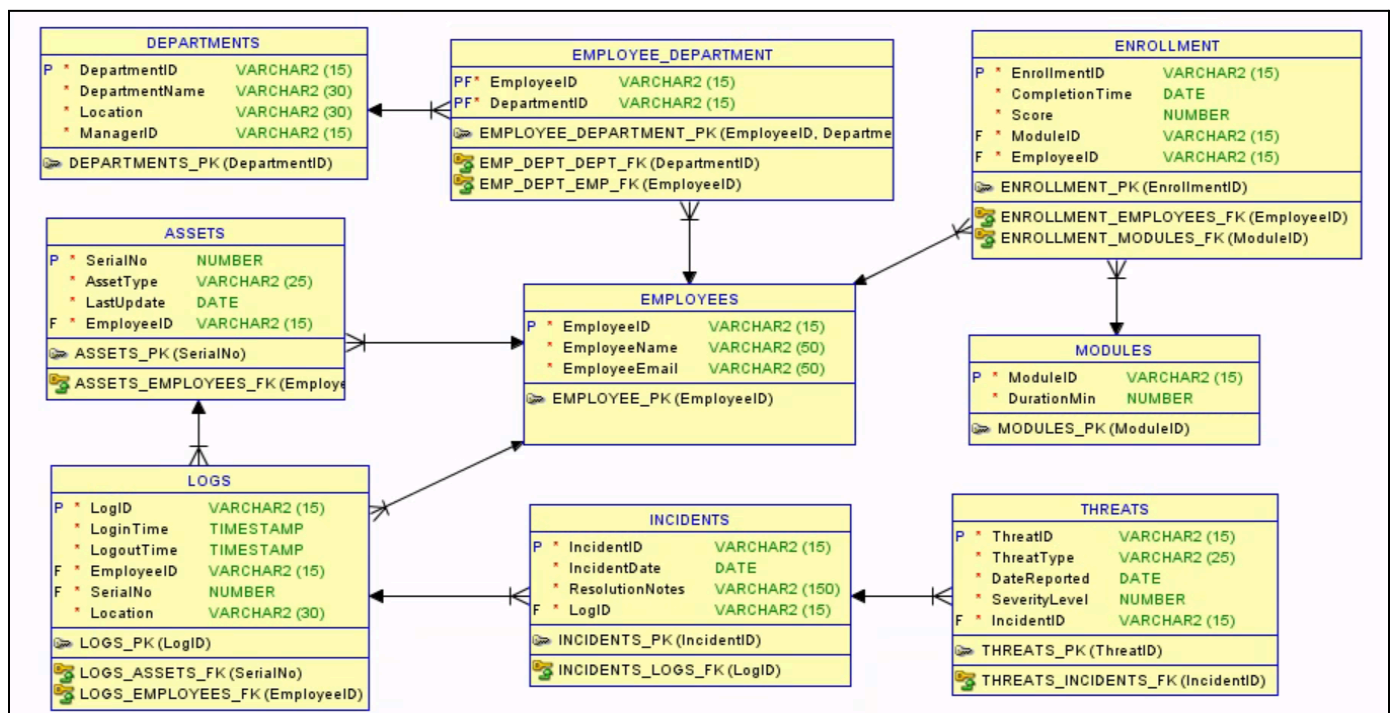
Revised Functional Dependency (FD) List:

- EMPLOYEES: EmployeeID → EmployeeName, EmployeeEmail
- DEPARTMENTS: DepartmentID → DepartmentName, Location, ManagerID
- ENROLLMENT: (EmployeeID, ModuleID) → CompletionDate, Score

Revised Logical Model



Physical Database Model



When it came to developing our relational database, we wanted to ensure that both attribute types and table relationships made complete sense, because this model serves as the foundation for inserting & filtering our data. The majority of our attributes were denoted as VARCHAR2 values of some capacity. Any ID attribute had a value of 15 characters, which allowed us to connect the tables on these primary

keys with no issues. Our other attribute types were relatively self-explanatory; Login & logout times are denoted as TIMESTAMP, any date values are denoted as DATE, and values such as “SeverityLevel” and “DurationMin” are denoted as NUMBER.

Here are the following entities in our database:

If an attribute appears as a foreign key in another table, it is only described in its original table.

1. EMPLOYEES: Stores employee details and department links
 - a. EmployeeID: A unique identifier for each credit union employee;
 - b. EmployeeName: The full name of the employee
 - c. EmployeeEmail: The email address for security alerts and training module notifications.
2. DEPARTMENTS: Manages the different credit union departments
 - a. DepartmentID: A unique identifier for each department (e.g., IT, HR)
 - b. DepartmentName: The name of the department
 - c. Location: The branch or office location where the department is based
 - d. ManagerID: The EmployeeID of the individual responsible for overseeing this department.
3. EMPLOYEE_DEPARTMENT (Associative Entity)
 - a. EmployeeID: A foreign key linking to the Employees table
 - b. DepartmentID: A foreign key linking to the Departments table
4. ASSETS: Tracks computers/hardware
 - a. SerialNo: The unique serial number assigned by the manufacturer
 - b. AssetType: The type of hardware (e.g., laptop, mobile device)
 - c. LastUpdate: The timestamp of the most recent security fix or firmware update
 - d. EmployeeID: A foreign key identifying which employee the asset is currently assigned to.
5. THREATS: Log scams like deepfakes or phishing
 - a. ThreatID: A unique identifier for type of security risk
 - b. ThreatType: The category of the attack (e.g., Deepfake Voice, Phishing, Social Engineering)
 - c. DateReported: The date the threat was first identified in the system
 - d. SeverityLevel: A numeric scale (1–5) indicating the potential impact of the threat
 - e. IncidentID: A foreign key linking the threat to a recorded incident
6. LOGS: Records every system access attempt and hardware association
 - a. LogID: A unique identifier used to track each individual system access session
 - b. LoginTime: The exact timestamp (date and time) when the employee successfully accessed the system
 - c. LogoutTime: The exact timestamp when the employee’s session was terminated or closed.
 - d. EmployeeID: A foreign key linking the log entry to an individual in the employee roster.
 - e. SerialNo: A foreign key identifying the specific hardware asset used by the employee during a session.
 - f. Location: The physical branch office (or network IP location) from which the login attempt originated.

7. INCIDENTS: Logs specific security events reported by employees
 - a. IncidentID: A unique identifier for a confirmed security breach or event
 - b. IncidentDate: The date the security event occurred
 - c. ResolutionNotes: A descriptive narrative of how the incident was handled and resolved.
 - d. LogID: A foreign key linking the incident back to the specific system log where the activity was caught.
8. MODULES: Lists the available security training modules
 - a. ModuleID: A unique identifier for a specific security training module
 - b. DurationMin: The estimated time in minutes required to complete the module
9. ENROLLMENT: Tracks which employees have completed which training modules
 - a. EnrollmentID: A unique identifier for a specific training attempt
 - b. CompletionDate: The date the employee successfully finished the module
 - c. Score: The (numerical) grade achieved by the employee on the module
 - d. EmployeeID: A foreign key identifying the employee taking the module
 - e. ModuleID: A foreign key identifying which specific module is being taken

Operational SQL

1. Reporting High-Risk Training (Subquery)

```
SELECT EmployeeID, EmployeeName, EmployeeEmail
FROM EMPLOYEES
WHERE EmployeeID IN (
    SELECT EmployeeID
    FROM ENROLLMENT
    GROUP BY EmployeeID
    HAVING MAX(Score) < (SELECT AVG(Score) FROM ENROLLMENT));
```

A manager needs to know which employees are falling behind in cybersecurity readiness. This subquery identifies employees whose highest training score is lower than the company-wide average. By isolating these employees, managers can provide targeted 1-on-1 mentoring or assign additional deepfake simulation tests to mitigate risk.

2. Number of Incidents by Location (Inner Join Query)

```
SELECT LOGS.Location, COUNT(INCIDENTS.IncidentID) AS
Total_Incidents
FROM LOGS
INNER JOIN INCIDENTS
ON LOGS.LogID = INCIDENTS.LogID
GROUP BY LOGS.Location
ORDER BY Total_Incidents DESC;
```

A manager might want to identify locations experiencing the highest frequency of security incidents. This inner join query identifies specific branches requiring urgent retraining to mitigate future risks. Also, it assists the IT department in determining highly targeted sites, allowing for defensive measures to be implemented before an isolated incident escalates into a more significant threat.

3. Operational Asset Audit (Outer Join Query)

```
SELECT ASSETS.SerialNo, ASSETS.AssetType, ASSETS.LastUpdate,
MAX(LOGS.LoginTime) AS LastLogin
FROM ASSETS
LEFT OUTER JOIN LOGS ON ASSETS.SerialNo = LOGS.SerialNo
GROUP BY ASSETS.SerialNo, ASSETS.AssetType, ASSETS.LastUpdate
ORDER BY LastLogin DESC;
```

Managers must see that every asset is being actively used and monitored. This outer join query provides a complete inventory of all credit union hardware, displaying the most recent activity for each. By including the MAX (LOGS.LoginTime), the IT department can distinguish between active and inactive devices. For assets with LastLogin is NULL, managers can identify equipment that may have been lost, stolen, or misplaced. This allows the Credit Union to remove unneeded hardware—reducing the physical attack surface and ensuring that every active device is accounted for.

4. Training Module Performance Summary (GROUP BY w/o HAVING Query)

```
SELECT MODULES.ModuleID,
    COUNT(ENROLLMENT.EmployeeID) AS EmployeesEnrolled,
    ROUND(AVG(ENROLLMENT.Score), 2) AS AverageScore
FROM MODULES
LEFT JOIN ENROLLMENT
ON MODULES.ModuleID = ENROLLMENT.ModuleID
GROUP BY MODULES.ModuleID
```

```
ORDER BY AverageScore DESC;
```

While this query is simple, it provides managers with a look into how many employees have taken a module & the average score that they received. Since this query doesn't have the HAVING clause, it's more of a summary, and it groups the data by module.

5. Risk Identification Report (GROUP BY + HAVING Query)

```
SELECT EMPLOYEES.EmployeeID, EMPLOYEES.EmployeeName,  
COUNT(INCIDENTS.IncidentID) AS TotalIncidents  
FROM EMPLOYEES  
JOIN LOGS ON EMPLOYEES.EmployeeID = LOGS.EmployeeID  
JOIN INCIDENTS ON LOGS.LogID = INCIDENTS.LogID  
GROUP BY Employees.EmployeeID, Employees.EmployeeName  
HAVING COUNT(INCIDENTS.IncidentID) >= 1  
ORDER BY TotalIncidents DESC;
```

This GROUP BY with HAVING query would be able to identify employees that have produced an incident for the credit union. In identifying these employees, we would be able to facilitate discussions, spot any trend in employee-incident frequency, and proceed to check their test scores or what assets they use. This query is the starting point for getting a full picture of what's going on within our organization.

Relevant INDEXES:

```
Create Index idx_threats_severity on THREATS(SeverityLevel);
```

This column is filtered consistently and this would allow for any high level threat below a severity level of 4 to not have to do a full table scan to find relevant data.

```
Create Index idx_logs_employeeid on LOGS(EmployeeID);
```

The Logs table acts as a junction table with most tables joining on EmployeeID. Creating an index speeds up lookup times whenever testing things like tracing an incident back to an employee.

```
Create Index IDX_enrollment_employeeid on ENROLLMENT(EmployeeID);
```

As the enrollment table grows every time an employee completes training modules. This table can often be queried to group this table by employeeID so as more employees are hired this query would slow down without an index. This index speeds up the lookup process on EmployeeID so training reports remain up to speed even as the table expands.

Relevant VIEW: Employee Incident Tracker

```
CREATE OR REPLACE VIEW Employee_Incidents AS  
SELECT  
    EMPLOYEES.EmployeeID,  
    EMPLOYEES.EmployeeName,  
    COUNT(INCIDENTS.IncidentID) AS Incident_Count  
FROM EMPLOYEES  
LEFT JOIN LOGS ON EMPLOYEES.EmployeeID = LOGS.EmployeeID  
LEFT JOIN INCIDENTS ON LOGS.LogID = INCIDENTS.LogID  
GROUP BY  
    EMPLOYEES.EmployeeID,  
    EMPLOYEES.EmployeeName;
```

This view allows a manager to see the total number of incidents each employee has had. This will offer the manager the capability to see which employees are at a heightened risk for cyber attacks. This count will also illustrate to the manager which employees need module training the most.

Relevant TRIGGER: Incident Occurrence Notification

```
CREATE OR REPLACE TRIGGER TRG_INCIDENT_AUDIT
AFTER INSERT ON INCIDENTS
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('New incident recorded');
    DBMS_OUTPUT.PUT_LINE(:NEW.IncidentID);
    DBMS_OUTPUT.PUT_LINE(:NEW.LogID);
END;
/
```

The purpose of this trigger is to notify the IT department whenever the database receives an insert in the incident section. This is important because it offers a real-time input whenever an incident occurs. This saves valuable time for management or the IT department. This trigger paste that a new incident is recorded, the incident ID, and the login ID of the incident. This occurs after the incident is inserted into SQL.

Deployment

SQL Agent

WEGZ Credit Union - Employee Portal

Start a chat. We're here to help you 24/7.

Welcome to WEGZ Credit Union Employee Portal!

How can I assist you today?

Type your query here...

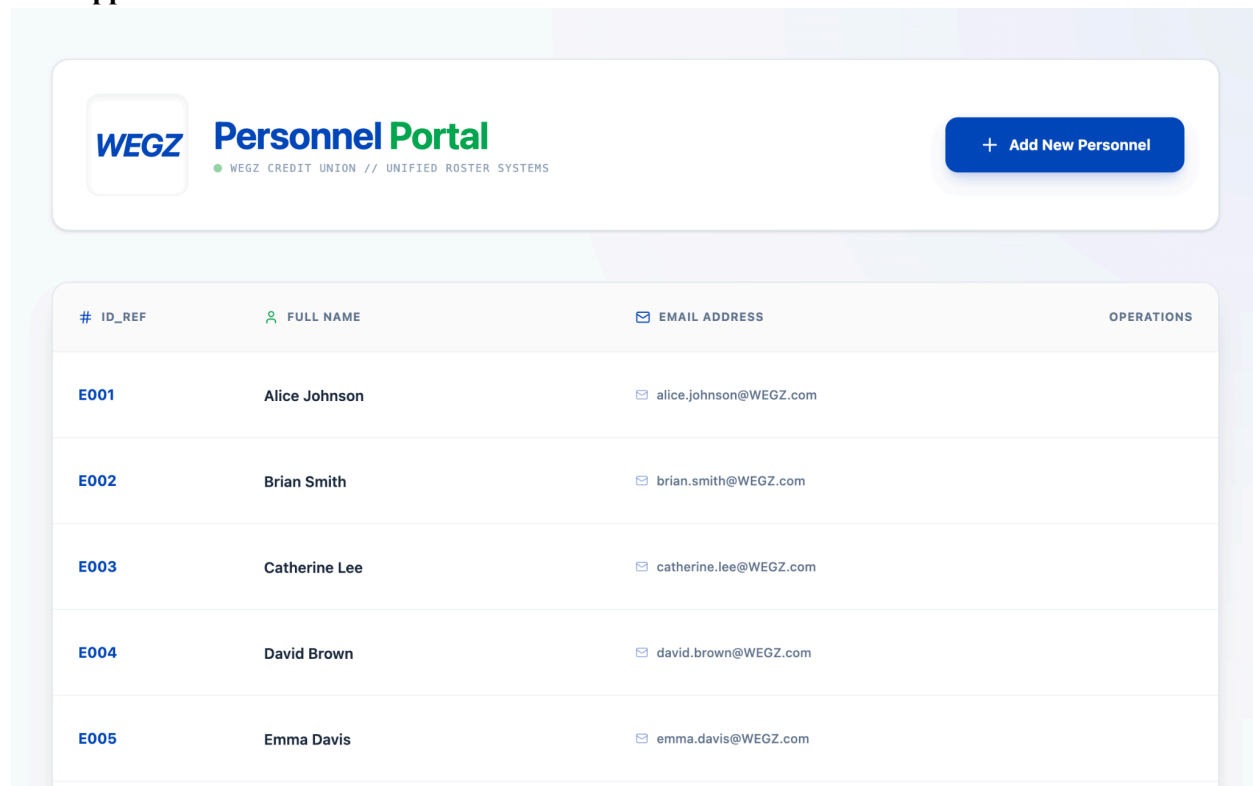


<https://miamiuniversity.app.n8n.cloud/webhook/51cd1a5a-f9e5-4b60-a983-746dca0791e3/chat>

SQL Agent Value:

The agent acts as the database's proactive security guard. While the web app is used for manual entry, the agent runs in the background 24/7 to monitor the employee roster and incident logs. It eliminates manual oversight by automatically identifying employees who are overdue for deepfake training or marking assets that haven't had a security update. By using natural language processing, it allows managers to "ask" the database questions in English rather than writing SQL code. The agent ensures that leadership makes fast, informed decisions by putting out alerts about high-severity threats (i.e., Level 4 or 5) directly to the IT department the moment they are logged, rather than waiting for someone to run a report.

Web Application



# ID_REF	FULL NAME	EMAIL ADDRESS	OPERATIONS
E001	Alice Johnson	alice.johnson@WEGZ.com	
E002	Brian Smith	brian.smith@WEGZ.com	
E003	Catherine Lee	catherine.lee@WEGZ.com	
E004	David Brown	david.brown@WEGZ.com	
E005	Emma Davis	emma.davis@WEGZ.com	

<https://ai.studio/apps/c90be3dc-60e6-49d9-8470-cd109b97ded5>

Web Application Value:

The web application serves as the primary administrative portal for the “Security Shield” employee roster. While the agen handles background monitoring, the app provides the essential interface for active data management and real-time oversight. It streamlines database operations by allowing managers to add new hires, updating department transfers, or remove employees through a simple, user-friendly dashboard. The application prevents the security gaps that occur when using disconnected spreadsheets. The app allows those with non-technical backgrounds to visualize training compliance and asset distribution at a glance, turning data into insights. The app ensures that maintaining the credit union’s security position is an efficient, daily habit rather than a technical burden.

AI Process

Turning our database into a web application and assigning it an SQL agent was the final step of our project and after working through some minor inconveniences turned into a neat concept. The first step for both of these applications was uploading our data to the cloud database. We had multiple issues when uploading the data via a DDL file so rather we waited until we were confident that no more changes would be made to our create table and insert statements. Once we had completed both we uploaded them with zero issues and began working on our SQL agent.

Our SQL agent creation was relatively smooth. We have six key processes that take place in order for it to run which includes the initial prompt portal, simple memory, AI agent (we used GPT-5-Mini which works well for the most part with the occasional error message), and two SQL tools to help the agent sort through the data and access it. We did not talk about simple memory in class but after learning about it during our time in the project we believed it would be helpful so the user could callback to conversations previously mentioned in the chat. After some sample tests and adding some additional notes

to the system message we were able to get a SQL agent that can perform queries between multiple tables and in complex situations although it can take a minute to perform them.

The next step was the web application. We decided the best use of our web application was to make it a hub to overview current employees, add new employees, and delete employees. This delegates responsibilities between both the SQL agent and web application, the web application creating employees with an easy to use interface and the SQL agent diving into specifics such as what department, start date, etc. The site is color coded based on our company logo and shows all active employees with the ability to edit information if need be.

Overall, we enjoyed this part of the project the most because it displayed the possibilities a database can administer using a real world example. AI is going to be the future of how databases are navigated and learning how the two connect gives us a real competitive advantage going into the workforce.

Conclusions & Lessons Learned

Lessons & Conclusions

In partnership with *WEGZ Credit Union*, our team has successfully deployed a database that manages all electronic safety education and training. Along with the training and education, our database manages cyber threats throughout the Country. Our database has offered several benefits to the *WEGZ Credit Union*. This includes incident monitoring for the IT department and the ability to identify at-risk employees. A way to track which area experiences the most cyber attacks, and the ability to track employee logins to prevent unwanted access.

At the start of our work with *WEGZ Credit Union*, our team was tasked with developing a database to manage employee education and monitor potential threats to the business. From these needs, we developed a database that contained the following tables: employees, departments, logs, threats, incidents, modules, assets, and enrollment. The tables employees, departments, and logs were created to monitor employees and their locations to pick up on unusual patterns that management can identify and correct. The purpose of the enrollment and modules is to track employee cyber education and provide *WEGZ Credit Union* with useful data on the outcome of their employee-mandated cyber modules. For our threats, incidents, and asset tables, we use them to monitor what cyber attacks occur and on which devices they occur. This provides *WEGZ Credit Union* with valuable data it can use to continue to optimize its safety practices.

To ensure *WEGZ Credit Union* continues to optimize its electronic practices, we have created a trigger to alert the IT department or Management that an incident has occurred, this saves valuable time when identifying that day's incidents. We have created a view that notifies management which employees have had incident reports in the past, allowing management to identify which employees are at higher risk for attacks. As for other controls, we have developed both a website and an SQL agent that work alongside each other to update the database as new items are logged into the website. Controls on the website prevent employees from entering unfinished or incorrect data into the database. These include making specific identifiers unique and making others not null to ensure proper data is input.

While not exactly according to plan, our team has been able to develop a database that *WEGZ Credit Union* is happy with and believes will cut back on time costs that have been plaguing the company for the past few years. During this project, our team struggled to eliminate redundancies within the database. To properly achieve the goals, we had to eliminate certain entities within our database to ensure that only one identifier entity was needed to pull all relevant information within the table. Our team was very proud to put this database into a 3NF format to eliminate redundancies within the database, making our data easier to use for backend employees. Not only did we have issues removing redundancies, but we also had problems occur throughout our project.

Our main issue while working on this project was figuring out how to properly transfer our database layout into functional SQL code. To achieve this, we tried a plethora of options. Our first attempt was to turn it into a DDL file and use the code provided by it. Unfortunately, this code was plagued with errors all over, making it impossible to run the select statement. To remedy this issue, our team turned to Artificial Intelligence to provide a baseline of code that performed the functions we needed. Once the base code was developed, our team worked through it line by line to make sure each attribute of the table functioned like it was originally intended.

For changes in our original project, we removed smaller entities such as modulename and developed a table to track employees who may be a part of more than one department, named EMPLOYEE_DEPARTMENT. To fill our database, we intended to place 10 faux inserts into each table to display the functions of our database. After completing this task, we realized that this would make each of our employees managers, so our team chose to add another 5 faux inserts into our employee table to offer some balance to the team trying the product. Apart from these changes, our group did not have to deviate

much from our original plan. In conclusion, our team looks forward to seeing what *WEGZ Credit Union* can achieve with the help of our newly implemented database.

In Summary

Don't be like Canvas...

Copy/Paste Version Code

```

CREATE TABLE EMPLOYEES (
    EmployeeID VARCHAR2(15) PRIMARY KEY,
    EmployeeName VARCHAR2(50) NOT NULL,
    EmployeeEmail VARCHAR2(50) NOT NULL
);

CREATE TABLE DEPARTMENTS (
    DepartmentID VARCHAR2(15) PRIMARY KEY,
    DepartmentName VARCHAR2(30) NOT NULL,
    Location VARCHAR2(30) NOT NULL,
    ManagerID VARCHAR2(15) NOT NULL
);

CREATE TABLE MODULES (
    ModuleID VARCHAR2(15) PRIMARY KEY,
    DurationMin NUMBER NOT NULL
);

CREATE TABLE EMPLOYEE_DEPARTMENT (
    EmployeeID VARCHAR2(15),
    DepartmentID VARCHAR2(15),
    CONSTRAINT employee_department_pk PRIMARY KEY (EmployeeID,
DepartmentID),
    FOREIGN KEY (EmployeeID) REFERENCES EMPLOYEES(EmployeeID),
    FOREIGN KEY (DepartmentID) REFERENCES DEPARTMENTS(DepartmentID)
);

CREATE TABLE ENROLLMENT (
    EnrollmentID VARCHAR2(15) PRIMARY KEY,
    CompletionTime DATE,
    Score NUMBER,
    ModuleID VARCHAR2(15),
    EmployeeID VARCHAR2(15),
    FOREIGN KEY (ModuleID) REFERENCES MODULES(ModuleID),
    FOREIGN KEY (EmployeeID) REFERENCES EMPLOYEES(EmployeeID)
);

CREATE TABLE ASSETS (
    SerialNo NUMBER PRIMARY KEY,
    AssetType VARCHAR2(25),
    LastUpdate DATE,
    EmployeeID VARCHAR2(15),
    FOREIGN KEY (EmployeeID) REFERENCES EMPLOYEES(EmployeeID)
);

CREATE TABLE LOGS (

```

```
LogID VARCHAR2(15) PRIMARY KEY,  
LoginTime TIMESTAMP,  
LogoutTime TIMESTAMP,  
EmployeeID VARCHAR2(15),  
SerialNo NUMBER,  
Location VARCHAR2(30),  
FOREIGN KEY (EmployeeID) REFERENCES EMPLOYEES(EmployeeID),  
FOREIGN KEY (SerialNo) REFERENCES ASSETS(SerialNo)  
);
```

```
CREATE TABLE INCIDENTS (  
    IncidentID VARCHAR2(15) PRIMARY KEY,  
    IncidentDate DATE,  
    ResolutionNotes VARCHAR2(150),  
    LogID VARCHAR2(15),  
    FOREIGN KEY (LogID) REFERENCES LOGS(LogID)  
);
```

```
CREATE TABLE THREATS (  
    ThreatID VARCHAR2(15) PRIMARY KEY,  
    ThreatType VARCHAR2(25),  
    DateReported DATE,  
    SeverityLevel NUMBER,  
    IncidentID VARCHAR2(15),  
    FOREIGN KEY (IncidentID) REFERENCES INCIDENTS(IncidentID)  
);
```

```
INSERT INTO EMPLOYEES VALUES ('E001','Alice  
Johnson','alice.johnson@WEGS.com');  
INSERT INTO EMPLOYEES VALUES ('E002','Brian  
Smith','brian.smith@WEGS.com');  
INSERT INTO EMPLOYEES VALUES ('E003','Catherine  
Lee','catherine.lee@WEGS.com');  
INSERT INTO EMPLOYEES VALUES ('E004','David  
Brown','david.brown@WEGS.com');  
INSERT INTO EMPLOYEES VALUES ('E005','Emma  
Davis','emma.davis@WEGS.com');  
INSERT INTO EMPLOYEES VALUES ('E006','Frank  
Wilson','frank.wilson@WEGS.com');  
INSERT INTO EMPLOYEES VALUES ('E007','Grace  
Taylor','grace.taylor@WEGS.com');  
INSERT INTO EMPLOYEES VALUES ('E008','Henry  
Anderson','henry.anderson@WEGS.com');  
INSERT INTO EMPLOYEES VALUES ('E009','Isabella  
Martinez','isabella.martinez@WEGS.com');
```

```
INSERT INTO EMPLOYEES VALUES ('E010','James
Thompson','james.thompson@WEGS.com');
INSERT INTO EMPLOYEES VALUES ('E011','Olivia
White','olivia.white@WEGS.com');
INSERT INTO EMPLOYEES VALUES ('E012','Liam
Harris','liam.harris@WEGS.com');
INSERT INTO EMPLOYEES VALUES ('E013','Sophia
Clark','sophia.clark@WEGS.com');
INSERT INTO EMPLOYEES VALUES ('E014','Noah
Lewis','noah.lewis@WEGS.com');
INSERT INTO EMPLOYEES VALUES ('E015','Ava
Walker','ava.walker@WEGS.com');
```

```
INSERT INTO DEPARTMENTS VALUES ('D001','Information
Security','Columbus, OH','E001');
INSERT INTO DEPARTMENTS VALUES ('D002','Risk Management','Cincinnati,
OH','E002');
INSERT INTO DEPARTMENTS VALUES ('D003','Compliance','Cleveland,
OH','E003');
INSERT INTO DEPARTMENTS VALUES ('D004','Operations','Dayton,
OH','E004');
INSERT INTO DEPARTMENTS VALUES ('D005','Training','Toledo,
OH','E005');
INSERT INTO DEPARTMENTS VALUES ('D006','Fraud Prevention','Columbus,
OH','E006');
INSERT INTO DEPARTMENTS VALUES ('D007','Internal Audit','Akron,
OH','E007');
INSERT INTO DEPARTMENTS VALUES ('D008','Security Ops','Canton,
OH','E008');
INSERT INTO DEPARTMENTS VALUES ('D009','Data Privacy','Cincinnati,
OH','E009');
INSERT INTO DEPARTMENTS VALUES ('D010','Incident Response','Columbus,
OH','E010');
```

```
INSERT INTO MODULES VALUES ('MOD001',45);
INSERT INTO MODULES VALUES ('MOD002',30);
INSERT INTO MODULES VALUES ('MOD003',60);
INSERT INTO MODULES VALUES ('MOD004',20);
INSERT INTO MODULES VALUES ('MOD005',15);
INSERT INTO MODULES VALUES ('MOD006',40);
INSERT INTO MODULES VALUES ('MOD007',25);
INSERT INTO MODULES VALUES ('MOD008',50);
INSERT INTO MODULES VALUES ('MOD009',35);
INSERT INTO MODULES VALUES ('MOD010',20);
```

```
INSERT INTO EMPLOYEE_DEPARTMENT VALUES ('E001','D001');
```

```
INSERT INTO EMPLOYEE_DEPARTMENT VALUES ('E002','D002');
INSERT INTO EMPLOYEE_DEPARTMENT VALUES ('E003','D003');
INSERT INTO EMPLOYEE_DEPARTMENT VALUES ('E004','D004');
INSERT INTO EMPLOYEE_DEPARTMENT VALUES ('E005','D005');
INSERT INTO EMPLOYEE_DEPARTMENT VALUES ('E006','D006');
INSERT INTO EMPLOYEE_DEPARTMENT VALUES ('E007','D007');
INSERT INTO EMPLOYEE_DEPARTMENT VALUES ('E008','D008');
INSERT INTO EMPLOYEE_DEPARTMENT VALUES ('E009','D009');
INSERT INTO EMPLOYEE_DEPARTMENT VALUES ('E010','D010');
```

```
INSERT INTO ENROLLMENT VALUES ('EN001', DATE '2026-04-01', 85,
'MOD001', 'E001');
INSERT INTO ENROLLMENT VALUES ('EN002', DATE '2026-04-02', 92,
'MOD002', 'E002');
INSERT INTO ENROLLMENT VALUES ('EN003', DATE '2026-04-03', 78,
'MOD003', 'E003');
INSERT INTO ENROLLMENT VALUES ('EN004', DATE '2026-04-04', 88,
'MOD004', 'E004');
INSERT INTO ENROLLMENT VALUES ('EN005', DATE '2026-04-05', 95,
'MOD005', 'E005');
INSERT INTO ENROLLMENT VALUES ('EN006', DATE '2026-04-06', 73,
'MOD006', 'E006');
INSERT INTO ENROLLMENT VALUES ('EN007', DATE '2026-04-07', 81,
'MOD007', 'E007');
INSERT INTO ENROLLMENT VALUES ('EN008', DATE '2026-04-08', 90,
'MOD008', 'E008');
INSERT INTO ENROLLMENT VALUES ('EN009', DATE '2026-04-09', 76,
'MOD009', 'E009');
INSERT INTO ENROLLMENT VALUES ('EN010', DATE '2026-04-10', 89,
'MOD010', 'E010');
```

```
INSERT INTO ASSETS VALUES (1001,'Laptop',DATE '2026-01-15','E001');
INSERT INTO ASSETS VALUES (1002,'Workstation',DATE
'2026-02-03','E002');
INSERT INTO ASSETS VALUES (1003,'Laptop',DATE '2025-11-20','E003');
INSERT INTO ASSETS VALUES (1004,'Server',DATE '2026-03-10','E004');
INSERT INTO ASSETS VALUES (1005,'Laptop',DATE '2026-04-01','E005');
INSERT INTO ASSETS VALUES (1006,'Workstation',DATE
'2025-12-05','E006');
INSERT INTO ASSETS VALUES (1007,'Router',DATE '2026-01-28','E007');
INSERT INTO ASSETS VALUES (1008,'Laptop',DATE '2026-02-17','E008');
INSERT INTO ASSETS VALUES (1009,'Mobile Device',DATE
'2026-03-22','E009');
INSERT INTO ASSETS VALUES (1010,'Server',DATE '2026-04-05','E010');
```

```

INSERT INTO LOGS VALUES ('L001',TIMESTAMP '2026-04-20
08:14:00',TIMESTAMP '2026-04-20 09:02:00','E001',1001,'Columbus, OH');
INSERT INTO LOGS VALUES ('L002',TIMESTAMP '2026-04-21
07:55:00',TIMESTAMP '2026-04-21 08:47:00','E002',1002,'Cincinnati,
OH');
INSERT INTO LOGS VALUES ('L003',TIMESTAMP '2026-04-22
13:20:00',TIMESTAMP '2026-04-22 14:05:00','E003',1003,'Cleveland,
OH');
INSERT INTO LOGS VALUES ('L004',TIMESTAMP '2026-04-23
09:30:00',TIMESTAMP '2026-04-23 10:15:00','E004',1004,'Dayton, OH');
INSERT INTO LOGS VALUES ('L005',TIMESTAMP '2026-04-24
11:00:00',TIMESTAMP '2026-04-24 11:52:00','E005',1005,'Toledo, OH');
INSERT INTO LOGS VALUES ('L006',TIMESTAMP '2026-04-25
14:45:00',TIMESTAMP '2026-04-25 15:30:00','E006',1006,'Columbus, OH');
INSERT INTO LOGS VALUES ('L007',TIMESTAMP '2026-04-26
08:00:00',TIMESTAMP '2026-04-26 09:10:00','E007',1007,'Akron, OH');
INSERT INTO LOGS VALUES ('L008',TIMESTAMP '2026-04-27
10:20:00',TIMESTAMP '2026-04-27 11:05:00','E008',1008,'Canton, OH');
INSERT INTO LOGS VALUES ('L009',TIMESTAMP '2026-04-28
15:10:00',TIMESTAMP '2026-04-28 16:00:00','E009',1009,'Cincinnati,
OH');
INSERT INTO LOGS VALUES ('L010',TIMESTAMP '2026-04-29
09:45:00',TIMESTAMP '2026-04-29 10:30:00','E010',1010,'Columbus, OH');

```

```

INSERT INTO INCIDENTS VALUES ('I001',DATE '2026-04-20','Login
blocked','L001');
INSERT INTO INCIDENTS VALUES ('I002',DATE '2026-04-21','Malware
removed','L002');
INSERT INTO INCIDENTS VALUES ('I003',DATE '2026-04-22','Password
reset','L003');
INSERT INTO INCIDENTS VALUES ('I004',DATE '2026-04-23','Firewall
updated','L004');
INSERT INTO INCIDENTS VALUES ('I005',DATE '2026-04-24','Suspicious
email','L005');
INSERT INTO INCIDENTS VALUES ('I006',DATE '2026-04-25','Account
restored','L006');
INSERT INTO INCIDENTS VALUES ('I007',DATE '2026-04-26','VPN
issue','L007');
INSERT INTO INCIDENTS VALUES ('I008',DATE '2026-04-27','Security
patch','L008');
INSERT INTO INCIDENTS VALUES ('I009',DATE '2026-04-28','Account
disabled','L009');
INSERT INTO INCIDENTS VALUES ('I010',DATE '2026-04-29','System
patch','L010');

```

```

INSERT INTO THREATS VALUES ('T001','Phishing',DATE
'2026-04-20',1,'I001');
INSERT INTO THREATS VALUES ('T002','Malware',DATE
'2026-04-21',5,'I002');
INSERT INTO THREATS VALUES ('T003','Unauthorized Access',DATE
'2026-04-22',3,'I003');
INSERT INTO THREATS VALUES ('T004','Ransomware',DATE
'2026-04-23',5,'I004');
INSERT INTO THREATS VALUES ('T005','Data Breach',DATE
'2026-04-24',4,'I005');
INSERT INTO THREATS VALUES ('T006','Insider Threat',DATE
'2026-04-25',2,'I006');
INSERT INTO THREATS VALUES ('T007','DDoS Attack',DATE
'2026-04-26',5,'I007');
INSERT INTO THREATS VALUES ('T008','Spyware',DATE
'2026-04-27',3,'I008');
INSERT INTO THREATS VALUES ('T009','Brute Force Attack',DATE
'2026-04-28',4,'I009');
INSERT INTO THREATS VALUES ('T010','Trojan Horse',DATE
'2026-04-29',5,'I010');

```

```

SELECT EmployeeID, EmployeeName, EmployeeEmail
FROM EMPLOYEES
WHERE EmployeeID IN (
    SELECT EmployeeID
    FROM ENROLLMENT
    GROUP BY EmployeeID
    HAVING MAX(Score) < (SELECT AVG(Score) FROM ENROLLMENT));

```

```

SELECT LOGS.Location,COUNT(INCIDENTS.IncidentID) AS
Total_Incidents
FROM LOGS
INNER JOIN INCIDENTS
ON LOGS.LogID = INCIDENTS.LogID
GROUP BY LOGS.Location
ORDER BY Total_Incidents DESC;

```

```

SELECT ASSETS.SerialNo, ASSETS.AssetType, ASSETS.LastUpdate,
MAX(LOGS.LoginTime) AS LastLogin
FROM ASSETS
LEFT OUTER JOIN LOGS ON ASSETS.SerialNo = LOGS.SerialNo
GROUP BY ASSETS.SerialNo, ASSETS.AssetType, ASSETS.LastUpdate
ORDER BY LastLogin DESC;

```

```

SELECT MODULES.ModuleID,
COUNT(ENROLLMENT.EmployeeID) AS EmployeesEnrolled,

```

```

        ROUND(AVG(ENROLLMENT.Score), 2) AS AverageScore
FROM MODULES
LEFT JOIN ENROLLMENT
    ON MODULES.ModuleID = ENROLLMENT.ModuleID
GROUP BY MODULES.ModuleID
ORDER BY AverageScore DESC;

```

```

SELECT EMPLOYEES.EmployeeID, EMPLOYEES.EmployeeName,
COUNT(INCIDENTS.IncidentID) AS TotalIncidents
FROM EMPLOYEES
JOIN LOGS ON EMPLOYEES.EmployeeID = LOGS.EmployeeID
JOIN INCIDENTS ON LOGS.LogID = INCIDENTS.LogID
GROUP BY Employees.EmployeeID, Employees.EmployeeName
HAVING COUNT(INCIDENTS.IncidentID) >= 1
ORDER BY TotalIncidents DESC;

```

```

Create Index idx_threats_severity on THREATS(SeverityLevel);

```

```

Create Index idx_logs_employeeid on LOGS(EmployeeID);

```

```

Create Index IDX_enrollment_employeeid on ENROLLMENT(EmployeeID);

```

```

CREATE OR REPLACE VIEW Employee_Incidents AS
SELECT
    EMPLOYEES.EmployeeID,
    EMPLOYEES.EmployeeName,
    COUNT(INCIDENTS.IncidentID) AS Incident_Count
FROM EMPLOYEES
LEFT JOIN LOGS ON EMPLOYEES.EmployeeID = LOGS.EmployeeID
LEFT JOIN INCIDENTS ON LOGS.LogID = INCIDENTS.LogID
GROUP BY
    EMPLOYEES.EmployeeID,
    EMPLOYEES.EmployeeName;

```

```

CREATE OR REPLACE TRIGGER TRG_INCIDENT_AUDIT
AFTER INSERT ON INCIDENTS
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('New incident recorded');
    DBMS_OUTPUT.PUT_LINE(:NEW.IncidentID);
    DBMS_OUTPUT.PUT_LINE(:NEW.LogID);
END;
/

```